

Operating Systems: Abstractions, Mechanisms, and Execution

From privilege boundaries to virtual memory, concurrency, and isolation

SDB

Autumn 2026

Intermediate/graduate engineering lecture set

Learning outcomes

By the end of this lecture set, you should be able to:

1. distinguish an OS **abstraction**, **mechanism**, and **policy**;
2. trace execution across user mode, kernel mode, interrupts, and scheduling;
3. reason about processes, threads, synchronization, and deadlock;
4. explain address translation, page faults, copy-on-write, and memory pressure;
5. compare filesystems, containers, virtual machines, and distributed storage; and
6. evaluate designs using correctness, performance, security, and reliability trade-offs.

Guiding question

What must the OS virtualize, arbitrate, and protect so that mutually distrustful programs can safely share one machine?

Road map

- 1 OS Role and Architecture
- 2 Privilege, System Calls, and Execution
- 3 Scheduling and Real-Time Analysis
- 4 Synchronization and Concurrency
- 5 Virtual Memory and Memory Management
- 6 Storage and Distributed Filesystems
- 7 Isolation, Containers, and Virtual Machines
- 8 Diagnosis and Synthesis

Road map

- 1 OS Role and Architecture
- 2 Privilege, System Calls, and Execution
- 3 Scheduling and Real-Time Analysis
- 4 Synchronization and Concurrency
- 5 Virtual Memory and Memory Management
- 6 Storage and Distributed Filesystems
- 7 Isolation, Containers, and Virtual Machines
- 8 Diagnosis and Synthesis

The operating system contract

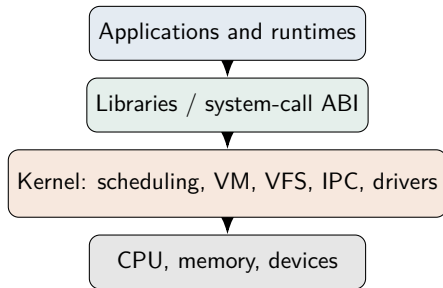
Three complementary views

- **Abstraction provider:** processes, virtual memory, files, sockets.
- **Resource manager:** allocates CPU, memory, I/O bandwidth, and energy.
- **Protection boundary:** mediates access between principals and devices.

Design discipline

Mechanism: “how?” **Policy:** “which/when?”

Example: a timer interrupt enables preemption; a scheduling policy chooses the next thread.



Architecture: where should services run?

	Monolithic/modular	Microkernel	Hybrid / library OS
Placement	Most services in kernel; loadable modules	Minimal kernel; servers in user space	Pragmatic mix or application-linked services
Strength	Fast in-kernel calls; mature ecosystem	Fault isolation; small trusted core	Tunable performance/isolation balance
Cost	Large privileged attack surface	IPC and crossing overhead; complex coordination	Architectural complexity; blurred boundaries
Examples	Linux, BSD	seL4, QNX, MINIX 3	Windows NT, XNU; uniker-nels

Graduate-level lens

Architecture is not a binary label. Modern kernels combine modules, user-space daemons, hardware isolation, verified components, and fast shared-memory IPC. Evaluate the *trusted computing base*, failure containment, and crossing cost.

Why historical categories are insufficient

Workload goals

- batch: throughput and utilization;
- interactive: latency and fairness;
- real-time: bounded worst-case response;
- mobile/embedded: energy, footprint, predictability;
- cloud: isolation, elasticity, observability.

Modern systems are combinations

- Linux supports batch, interactive, and real-time policies.
- A phone combines an application OS, secure enclave, and several RTOS-controlled devices.
- A cloud host combines a kernel, hypervisor features, containers, and distributed control planes.

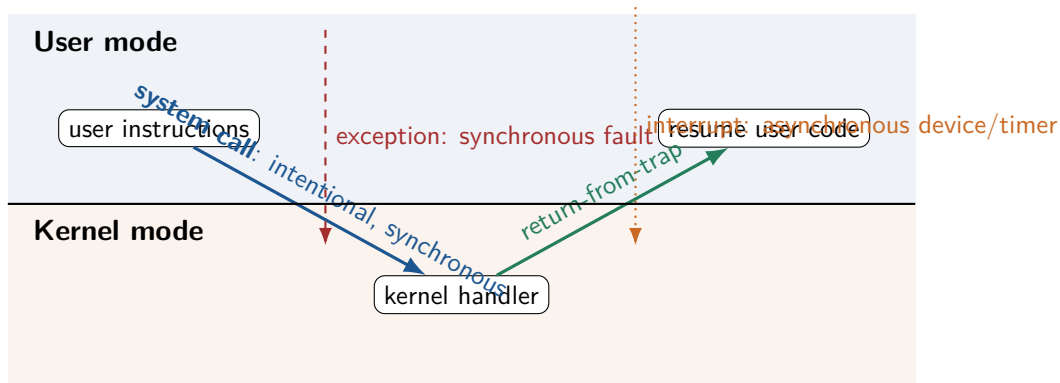
Precision

An RTOS does not magically guarantee every deadline. Guarantees require schedulable workloads, bounded interrupt/blocking times, suitable policies, and analyzable hardware/software paths.

Road map

- 1 OS Role and Architecture
- 2 Privilege, System Calls, and Execution**
- 3 Scheduling and Real-Time Analysis
- 4 Synchronization and Concurrency
- 5 Virtual Memory and Memory Management
- 6 Storage and Distributed Filesystems
- 7 Isolation, Containers, and Virtual Machines
- 8 Diagnosis and Synthesis

Privilege boundaries and control transfer



- Hardware saves a defined architectural state, switches privilege/stack as required, and vectors to a registered handler.
- The kernel validates arguments and authority; a system-call number is not permission.
- A mode switch **does not necessarily schedule another thread**.

System-call path: API, ABI, validation, result

1. Application calls a library API, e.g., `read(fd, buf, n)`.
2. A wrapper places the syscall number/arguments per the ABI and executes a trap instruction.
3. Entry code switches to a protected kernel stack and dispatches the handler.
4. Kernel checks descriptor validity, access rights, address ranges, and object state.
5. The call completes, blocks, or returns an error; the wrapper exposes the result/`errno`.

Three different events

Mode switch: privilege changes.

Context switch: running thread changes.

Address-space switch: translation context changes.

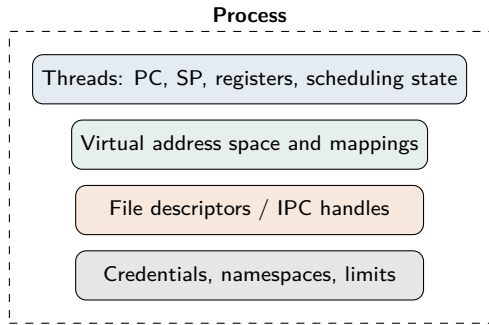
Fast paths

Some APIs avoid a trap: `vDSO` code can read kernel-published time data in user space; `io_uring` amortizes crossings through shared queues and batching.

Process: a protected execution environment

A process is more than “a program in execution.” It combines:

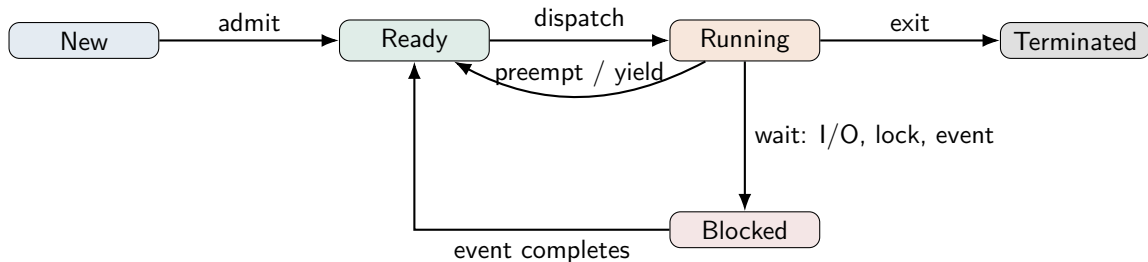
- a virtual address space and memory mappings;
- one or more schedulable threads;
- handles/file descriptors and IPC endpoints;
- credentials, namespaces, limits, and accounting;
- signal/disposition state and parent/child relationships.



Kernel representation

A process/thread control structure stores scheduling state, saved registers, kernel stack references, memory-map pointers, and resource metadata.

Thread state transitions



Important distinction

“Blocked” means not runnable until an event occurs. “Ready” means runnable but waiting for a CPU. A swapped-out or suspended state is an optional policy-specific refinement, not a universal base state.

Case study: launching gcc file.c

```
1 pid_t pid = fork();
2 if (pid == 0) {
3     execlp("gcc", "gcc", "file.c", NULL);
4     _exit(127);           // exec failed
5 }
6 int status;
7 while (waitpid(pid, &status, 0) < 0 &&
8     errno == EINTR) { }
```

1. `fork()` creates a child with a logically copied address space, normally implemented using **copy-on-write**.
2. `execve()` replaces the child's mappings and initializes a new program image; it does not create a process.
3. The loader maps the executable/interpreter and prepares arguments, environment, stack, and entry state.
4. `waitpid()` blocks the shell until child state changes, then reaps termination metadata.

What the scheduler does

It schedules runnable threads throughout this sequence; scheduling is not a single stage “after exec.” The compiler may block repeatedly on page faults and file I/O.

Threads, parallelism, and multicore reality

Threads in one process usually share

- code, heap, memory mappings;
- open-file table and process credentials;
- failure fate: memory corruption is process-wide.

Each thread retains

- registers, program counter, stack;
- scheduling and signal-mask state;
- thread-local storage.

Concurrency versus parallelism

Concurrency: multiple activities overlap in time.

Parallelism: activities execute simultaneously on different processing units.

Multicore complications

Cache coherence, NUMA placement, false sharing, memory ordering, interrupt affinity, and lock contention can dominate algorithmic work.

Context switching: state and cost

Potentially saved/restored

- program counter, stack pointer, general registers, flags;
- floating-point/vector state, often lazily or conditionally;
- kernel stack and accounting/debug state;
- address-space identifier/page-table root when changing processes.

Cost is not only instructions

- hot cache lines are displaced;
- branch predictors and prefetch streams lose relevance;
- TLB effects depend on ASIDs/PCIDs and invalidations;
- migrations add cache/NUMA penalties.

Measurement rule

Avoid quoting one universal “context-switch time.” Measure the relevant path, workload, architecture, kernel configuration, and cache state; distinguish direct switching latency from indirect microarchitectural cost.

Road map

- 1 OS Role and Architecture
- 2 Privilege, System Calls, and Execution
- 3 Scheduling and Real-Time Analysis**
- 4 Synchronization and Concurrency
- 5 Virtual Memory and Memory Management
- 6 Storage and Distributed Filesystems
- 7 Isolation, Containers, and Virtual Machines
- 8 Diagnosis and Synthesis

Scheduling objectives and metrics

For job i with arrival a_i , first run s_i , completion c_i , and service time b_i :

response $R_i = s_i - a_i$, turnaround $T_i = c_i - a_i$, waiting $W_i = T_i - b_i$.

Common objectives

- low tail latency and good responsiveness;
- high throughput/utilization;
- fairness or proportional share;
- deadline satisfaction and predictability.

No single optimum

Objectives conflict. Minimizing mean turnaround can starve long jobs; fairness can reduce throughput; energy-aware consolidation can increase latency.

Classical policies and modern refinements

Policy	Idea	Strength	Failure mode / refinement
FCFS	arrival order	simple, low overhead	convoy effect
SJF/SRTF	shortest predicted work	minimizes mean completion time under assumptions	future burst unknown; starvation
Round robin	rotating time quantum	responsive, bounded turn wait	too-small quantum raises overhead
Priority	highest priority first	expresses importance	starvation; aging; inversion handling
Fair-share	allocate weighted CPU share	multi-tenant fairness	latency depends on runnable population
Multilevel feedback	infer behavior from history	adapts to interactive/CPU-bound tasks	gaming and tuning complexity

Production schedulers also account for multicore load balancing, affinity, topology, energy, bandwidth contention, and heterogeneous CPUs.

Real-time scheduling: guarantees require analysis

For periodic task $\tau_i = (C_i, T_i, D_i)$, C_i is worst-case execution time, T_i period, and D_i relative deadline.

Utilization

$$U = \sum_{i=1}^n \frac{C_i}{T_i}.$$

On one preemptive CPU with independent tasks and $D_i = T_i$, earliest-deadline-first is schedulable if $U \leq 1$ under the ideal model.

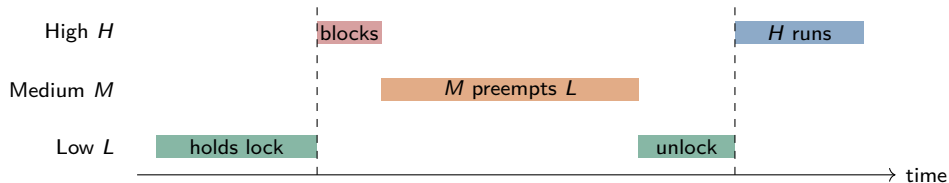
Reality adds

- blocking and non-preemptive regions;
- interrupt and kernel interference;
- cache/bus contention and execution-time variation;
- release jitter, overload, and multicore complications.

Hard versus soft real time

“Hard” means a missed deadline is unacceptable by specification; “soft” means lateness degrades utility. It does not mean “fast” versus “slow.”

Priority inversion and bounded blocking



- A high-priority thread waits for a lock held by a low-priority thread, while medium-priority work delays the lock holder.
- **Priority inheritance** temporarily raises the owner's effective priority; **priority-ceiling protocols** can provide analyzable blocking bounds.

Road map

- 1 OS Role and Architecture
- 2 Privilege, System Calls, and Execution
- 3 Scheduling and Real-Time Analysis
- 4 Synchronization and Concurrency**
- 5 Virtual Memory and Memory Management
- 6 Storage and Distributed Filesystems
- 7 Isolation, Containers, and Virtual Machines
- 8 Diagnosis and Synthesis

Race conditions and atomicity

```
1 // Incorrect if multiple threads execute it
2 counter++;
3
4 // Conceptual expansion
5 r = load(counter);
6 r = r + 1;
7 store(counter, r);
```

A **data race** occurs when conflicting accesses to the same location are concurrent, at least one is a write, and synchronization does not order them.

- Atomicity prevents torn/interleaved operations.
- Ordering determines which effects may be observed.
- Visibility requires communication through the memory model.

C/C++ caveat

A data race on a non-atomic object yields undefined behavior; “it works on my CPU” is not a correctness argument.

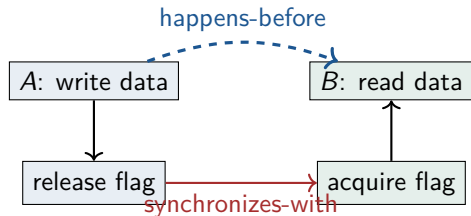
Happens-before: the useful abstraction

Reasoning rule

If event *A* **happens-before** event *B*, then *B* must observe the effects of *A* required by the language/OS synchronization semantics.

Typical ordering edges include:

- program order within one thread;
- mutex unlock $\rightarrow$ later successful lock;
- thread creation and join;
- release/acquire operations on compatible atomics.



Condition variables: wait in a loop

```
1 pthread_mutex_lock(&m);  
2 while (queue_empty(&q)) {           // predicate, not an event count  
3     pthread_cond_wait(&not_empty, &m);  
4     // atomically releases m while sleeping, then reacquires m  
5 }  
6 item = queue_pop(&q);  
7 pthread_mutex_unlock(&m);
```

Why the loop?

- wakeups may be spurious;
- another consumer may win the race;
- the predicate can become false before reacquisition.

Lock design

- keep invariants explicit;
- avoid blocking I/O while holding a lock;
- choose granularity using contention measurements;
- prefer structured ownership where possible.

Advanced synchronization toolbox

Primitive	Use	Key risk / insight
Mutex	exclusive access to an invariant	contention, convoying, priority inversion
Semaphore	counting resources or signaling	ownership is weaker; easy to mis-pair operations
Read–write lock	read-dominant shared state	writer starvation; overhead can exceed a mutex
Seqlock	many readers, rare short writes	readers retry; unsuitable for pointer reclamation alone
RCU	read-mostly structures	reclamation waits for grace periods; complex update semantics
Futex	user-space fast path, kernel-assisted sleep	basis for many Linux locks, not a policy itself
Lock-free atomics	system-wide progress despite a stalled thread	ABA, reclamation, memory-order complexity

Engineering principle

“Lock-free” is not synonymous with wait-free, faster, simpler, or starvation-free. State the progress guarantee and prove memory safety as well as atomic correctness.

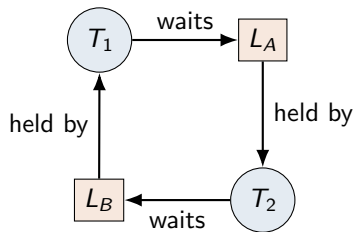
Deadlock: conditions, graph, and strategies

Four necessary Coffman conditions:

1. mutual exclusion;
2. hold and wait;
3. no forced preemption;
4. circular wait.

Design responses

Prevent one condition, avoid unsafe states, detect and recover, or accept the risk when justified.

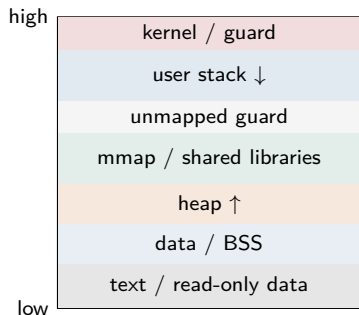


A cycle implies deadlock for single-instance resources; with multiple instances, a cycle is necessary but not always sufficient.

Road map

- 1 OS Role and Architecture
- 2 Privilege, System Calls, and Execution
- 3 Scheduling and Real-Time Analysis
- 4 Synchronization and Concurrency
- 5 Virtual Memory and Memory Management**
- 6 Storage and Distributed Filesystems
- 7 Isolation, Containers, and Virtual Machines
- 8 Diagnosis and Synthesis

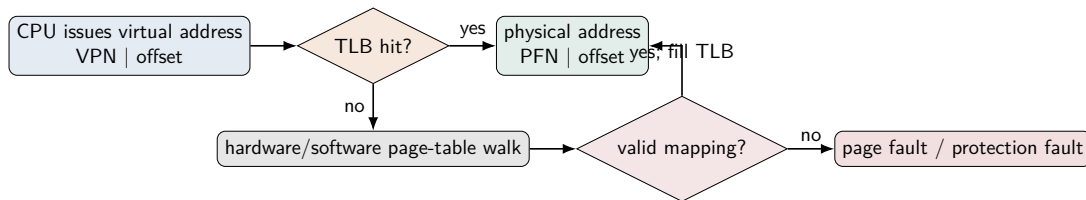
Virtual address space: isolation plus indirection



- Every process sees a private, sparse virtual address space; mappings need not occupy contiguous physical frames.
- Page permissions enforce read/write/execute and user/kernel access.
- Shared mappings permit efficient IPC and shared libraries.
- ASLR randomizes placement; guard pages turn some overflows into faults.
- The diagram is conceptual: exact layout depends on ABI, architecture, loader, and policy.

Virtual memory supplies protection and placement. Demand paging is a common policy layered on top.

Address translation and the TLB



- Multi-level page tables make sparse spaces practical; huge pages reduce TLB pressure but increase internal fragmentation and migration cost.
- A TLB miss is normally resolved by a page-table walk; it is **not automatically a page fault**.

Page faults: minor, major, and protection

1. CPU detects a missing or disallowed translation and traps to the kernel.
2. Kernel locates the virtual-memory area and checks requested access.
3. If valid, it may allocate a zero page, resolve copy-on-write, or fetch file/swap data.
4. Page tables and relevant TLB entries are updated; the faulting instruction restarts.

Minor fault

No storage I/O required; mapping/data is already resident or can be created cheaply.

Major fault

Requires storage I/O and can be orders of magnitude slower.

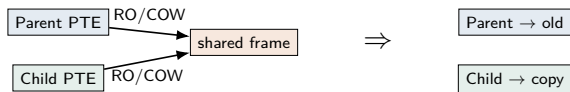
Invalid access

Kernel cannot resolve it; the process receives a fault signal/exception or is terminated.

Copy-on-write makes `fork()` practical

Immediately after `fork()`

- parent and child mappings reference the same physical frames;
- writable pages are marked read-only/COW;
- reference counts track sharing.



If the child quickly calls `execve()`, most pages are never copied. Page-table copying and shutdowns can still be significant for very large processes.

On a write fault

- allocate a new frame;
- copy the old contents;
- remap the writer's page as writable.

Memory pressure, replacement, and NUMA

Replacement goals

- approximate recency/frequency without exact LRU cost;
- balance anonymous and file-backed pages;
- preserve a workload's working set;
- write back dirty pages without bursty stalls.

Thrashing

If active working sets exceed available memory, page-fault work can dominate useful execution. Adding runnable jobs may reduce throughput.

NUMA-aware management

- local memory usually has lower latency/higher bandwidth;
- first-touch placement links allocation to the accessing CPU;
- migration trades locality benefit against movement cost;
- CPU affinity and memory placement must be considered together.

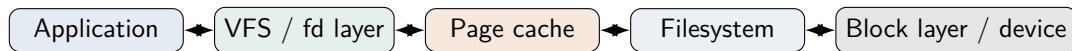
Modern concern

Memory bandwidth and cache capacity can be the scarce resources even when free pages and CPU time remain.

Road map

- 1 OS Role and Architecture
- 2 Privilege, System Calls, and Execution
- 3 Scheduling and Real-Time Analysis
- 4 Synchronization and Concurrency
- 5 Virtual Memory and Memory Management
- 6 Storage and Distributed Filesystems**
- 7 Isolation, Containers, and Virtual Machines
- 8 Diagnosis and Synthesis

From read() to persistent storage



- VFS provides a common namespace/object interface.
- The page cache unifies many buffered reads and memory-mapped file pages.
- Read-ahead and write-back convert application access into efficient I/O.
- `write()` returning does not generally imply durable media persistence.
- Durability requires defined APIs such as `fsync()` plus correct device/cache behavior.
- Direct I/O bypasses parts of caching but adds alignment and coordination constraints.

Crash consistency: preserving invariants

A crash can expose only a prefix or reordering of intended writes. The filesystem must prevent metadata/data structures from becoming inconsistent.

Journaling / WAL

Record intended updates in a log, commit them, then checkpoint to home locations.

Copy-on-write

Write new blocks and atomically redirect a root/reference; retain old reachable state.

Log-structured

Append updates sequentially; recover from checkpoints and log scanning.

Durability is end-to-end

Application update protocol, filesystem ordering, storage barriers, controller caches, and device semantics must agree. Atomic rename alone does not automatically make every multi-file transaction durable.

Distributed filesystems: semantics under failure

Design questions:

- Who owns metadata and namespace coordination?
- Are reads/writes linearizable, close-to-open, session-consistent, or eventually consistent?
- How are blocks replicated or erasure-coded?
- What happens during partitions, retries, duplicate RPCs, and client failure?
- How are leases, locks, caching, and recovery coordinated?

Failure-aware API

A timeout means “the client did not learn the result,” not necessarily “the operation did not occur.” Operations and retry protocols should be idempotent or carry unique request identities.

Different targets

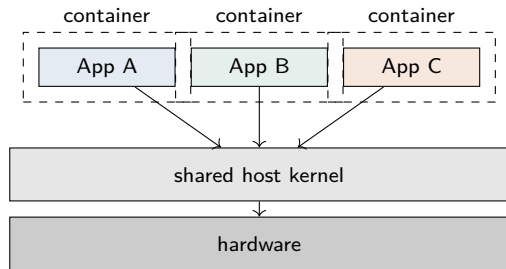
NFS-style sharing, parallel HPC filesystems, cloud object stores, and database storage expose different consistency and performance contracts.

Road map

- 1 OS Role and Architecture
- 2 Privilege, System Calls, and Execution
- 3 Scheduling and Real-Time Analysis
- 4 Synchronization and Concurrency
- 5 Virtual Memory and Memory Management
- 6 Storage and Distributed Filesystems
- 7 Isolation, Containers, and Virtual Machines**
- 8 Diagnosis and Synthesis

Containers are constrained processes, not tiny VMs

- **Namespaces** virtualize views of process IDs, mounts, networking, users, IPC, and host identity.
- **cgroups** account for and limit CPU, memory, I/O, and process counts.
- **Capabilities**, seccomp, LSMs, and filesystem permissions reduce authority.
- Layered images and runtimes package a user-space environment; all containers still share the host kernel.



Security boundary

Container isolation depends critically on the shared kernel and configuration; use stronger isolation where the threat model requires it.

Containers versus virtual machines

	Container	Virtual machine
Isolation layer	kernel objects/views and resource controls	virtual hardware plus guest kernel
Kernel	shared with host	separate guest kernel
Startup/footprint	usually small and fast	usually larger; snapshots can reduce startup
Compatibility	host kernel ABI required	can run a different compatible guest OS/ABI
Failure/security boundary	host kernel is common dependency	hypervisor/hardware boundary; still not absolute
Typical use	deployment density, reproducibility	stronger tenant separation, legacy OS, kernel control

Modern continuum

Sandboxed runtimes, microVMs, confidential VMs, and hardware enclaves combine techniques. Choose based on threat model, performance, compatibility, and operational cost.

Security: mediation, least privilege, defense in depth

Enforcement mechanisms

- privilege levels and page permissions;
- identities, ACLs, capabilities, and labels;
- syscall filtering and sandboxing;
- code signing, secure boot, measured boot;
- audit logs and anomaly detection.

Principles

- least privilege and least authority;
- complete mediation;
- minimize trusted computing base;
- fail safely and separate duties;
- assume components will be compromised.

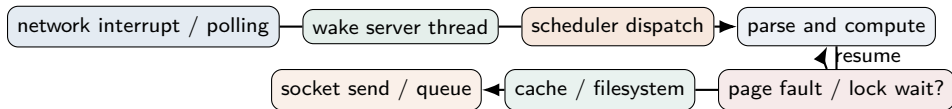
Side channels cross abstractions

Timing, caches, predictors, speculative execution, and shared resource contention can leak information even when architectural access checks are correct.

Road map

- 1 OS Role and Architecture
- 2 Privilege, System Calls, and Execution
- 3 Scheduling and Real-Time Analysis
- 4 Synchronization and Concurrency
- 5 Virtual Memory and Memory Management
- 6 Storage and Distributed Filesystems
- 7 Isolation, Containers, and Virtual Machines
- 8 **Diagnosis and Synthesis**

End-to-end execution: one request, many subsystems



Tail-latency lesson

A slow request may reflect scheduling delay, lock contention, reclaim, major faults, write-back, network queues, or device latency. Optimize only after identifying the dominant queue and causal path.

- Use traces for sequence/causality, profiles for where CPU time goes, and counters for rates/capacity symptoms.
- Coordinate IDs and timestamps across layers; averages often hide the failures users experience.

Common misconceptions: corrected

Misconception	⇒	Precise statement
Every syscall causes a context switch.	⇒	A syscall changes privilege; another thread runs only if scheduling occurs.
<code>fork()</code> immediately copies all memory.	⇒	It exposes copy semantics, usually via copy-on-write mappings.
A TLB miss is a page fault.	⇒	A valid page-table walk can resolve a TLB miss without a fault.
<code>write()</code> means data is durable.	⇒	It often means data reached kernel buffers; durability needs an explicit protocol.
Volatile makes shared variables thread-safe.	⇒	Language atomics or synchronization establish atomicity and ordering.
Containers each contain a kernel.	⇒	Ordinary containers share the host kernel; VMs run guest kernels.
RTOS means all deadlines are guaranteed.	⇒	Guarantees depend on workload assumptions and schedulability analysis.

Synthesis: the recurring design tensions

Performance

Cache locality, batching, fewer crossings, asynchronous I/O, scalable synchronization.

Correctness

Invariants, atomicity, ordering, bounded blocking, crash consistency, explicit semantics.

Isolation

Least privilege, resource controls, fault containment, auditable trusted components.

Transferable method

For any OS feature, ask: What abstraction is promised? Which mechanisms implement it? Which policies choose among alternatives? What assumptions can fail? How will we measure the result?

Discussion and next steps

1. Where should a database rely on the OS, and where should it bypass or specialize OS mechanisms?
2. Which shared resource is most likely to violate isolation in a multi-tenant service: CPU time, cache, memory bandwidth, storage, or network queues?
3. When is a stronger boundary (microVM/VM) worth its additional operational cost?

Suggested laboratory

Trace a small program using `strace`; inspect mappings with `/proc/<pid>/maps`; measure faults and switches with `perf stat`; then explain observed events using the models in this deck.

Road map

9 Appendix

Knowledge check

1. A thread calls `getpid()` through a vDSO implementation. Must it enter kernel mode?
2. A TLB lookup misses, but the page-table entry is present and accessible. Is this a page fault?
3. Why can `fork()` still be expensive for a process that immediately calls `execve()`?
4. Give one execution in which `if (empty) wait()` loses a wakeup or consumes invalid state.
5. What extra assumptions are needed before $U \leq 1$ establishes EDF schedulability?
6. After a distributed filesystem RPC times out, why is blind retry unsafe for a non-idempotent operation?

Exercise: analyze a deadline miss

A control task H with period/deadline 10 ms and WCET 2 ms shares a mutex with a logging task L . A telemetry task M can run for 6 ms. At $t = 0$, L holds the mutex; H releases and blocks; then M preempts L .

1. Draw the schedule without priority inheritance.
2. Identify direct blocking and interference.
3. Redraw it with priority inheritance.
4. What additional quantities are needed for a valid worst-case response-time bound?

Extension

Compare priority inheritance with a priority-ceiling protocol. Which better supports offline blocking analysis, and what implementation constraints follow?

Exercise: diagnose the concurrent queue

```
1 if (queue_empty(&q))  
2     pthread_cond_wait(&not_empty, &m);  
3 item = queue_pop(&q);
```

1. List every precondition the fragment silently assumes.
2. Construct an execution with two consumers that violates safety.
3. Repair the code and state the invariant protected by `m`.
4. Decide whether `signal` or `broadcast` is appropriate and justify the liveness/performance trade-off.

Further study

- Process/kernel interfaces: system-call ABI, signals, IPC, asynchronous I/O.
- Scheduling: multiprocessor fairness, deadline scheduling, energy-aware scheduling.
- Concurrency: memory models, RCU, lock-free reclamation, formal verification.
- Memory: huge pages, NUMA balancing, compressed memory, persistent memory.
- Storage: transactions, checksums, snapshots, replication, consensus-backed metadata.
- Isolation: capabilities, verified kernels, confidential computing, side-channel defenses.

Questions?